DevOps Infrastructure – Unit 2 Notes (Part 1)

Dockerfile Core Concepts and Image Building Basics

---

1. Dockerfile – Core Concept

Definition

A Dockerfile is a text file that contains a set of instructions used to automatically build a Docker image.

It defines:

- Base image
- Application dependencies
- Configuration
- Commands to run the application

Why Dockerfile is Important

- Automates image creation
- Ensures consistency across environments
- Enables version control of infrastructure
- Essential for CI/CD pipelines

---

2. Image Layering in Dockerfile

Concept

Each instruction in a Dockerfile creates a new layer in the image.

Example:

```
FROM ubuntu
RUN apt-get update
RUN apt-get install python3
COPY app.py /app/
```

Layers created:

1. Ubuntu base layer

2. Update packages
3. Install Python
4. Copy application

Benefits of Layering

- Faster builds using caching
- Reduced storage usage
- Easy reuse of common layers

---

3. Build Context

Definition

The build context is the set of files and directories that Docker can access during the build process.

When you run:
docker build .

The "." represents the current directory, which becomes the build context.

Important Point

- Docker can only access files inside the build context
- Files outside cannot be copied into the image

---

4. .dockerignore File

Definition

.dockerignore is used to exclude files and directories from the build context.

Why It is Needed

- Reduces build time
- Improves security (avoid sending sensitive files)
- Reduces image size

Example

.dockerignore file:

```
node_modules
.git
.env
*.log
```

---

5. Writing a Dockerfile (Basic Structure)

A typical Dockerfile follows this order:

1. Base image
2. Set working directory
3. Copy files
4. Install dependencies
5. Define environment variables
6. Specify default command

Example Dockerfile

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
CMD ["node", "app.js"]
```

---

6. Basic Dockerfile Instructions

6.1 FROM

- Specifies the base image
- Must be the first instruction

Example:
FROM ubuntu

6.2 RUN

- Executes commands during build time
- Used to install packages

Example:
RUN apt-get update && apt-get install -y python3

6.3 COPY

- Copies files from host to container

Example:
COPY app.py /app/

6.4 ADD

- Similar to COPY but with extra features:
    - Can extract compressed files
    - Can fetch remote URLs

Example:
ADD file.tar.gz /app/

Difference between COPY and ADD

- COPY is simple and preferred
- ADD has extra functionality but less predictable

6.5 CMD

- Specifies default command to run when container starts
- Can be overridden

Example:
CMD ["python3", "app.py"]

6.6 ENTRYPOINT

- Defines the main command of the container
- Cannot be easily overridden

Example:
ENTRYPOINT ["python3"]

CMD vs ENTRYPOINT

CMD	ENTRYPOINT
Can be overridden	Hard to override
Default command	Fixed executable

6.7 WORKDIR

- Sets working directory inside container

Example:
WORKDIR /app

6.8 ENV

- Sets environment variables

Example:
ENV PORT=5000

6.9 EXPOSE

- Declares port used by container

Example:
EXPOSE 3000

6.10 VOLUME

- Defines mount point for persistent storage

Example:
VOLUME /data

---

7. Summary

- Dockerfile is used to build images automatically
- Each instruction creates a layer
- Build context controls accessible files
- .dockerignore optimizes builds
- Core instructions include FROM, RUN, COPY, CMD, ENTRYPOINT, etc.